Quicksort Implementation, Analysis, and Randomization

Furqaanahmed Khan

University of the Cumberlands

MSCS 532: Algorithms and Data Structures

Professor Michael Solomon

November 15, 2026

Quicksort Implementation, Analysis, and Randomization

Quicksort divides a list around a pivot and sorts the resulting subarrays. This implementation compares a fixed last-element pivot with a randomly chosen pivot, using the same partition procedure for both. The measurements illustrate the effect of input order and show why randomization improves expected performance without guaranteeing a fast run.

Implementation and Complexity

Each partition moves the selected pivot to the end, scans the remaining active subarray, and places values less than or equal to the pivot on its left. A final exchange puts the pivot in its sorted position; larger values are to its right. The algorithm recursively sorts the smaller side and iterates over the larger side. This produces the same subproblems as recursive Quicksort while keeping the recursion stack shallow. Both functions modify and return the original list; exchanges can reverse equal items, so the sort is not stable.

Partitioning a subarray of size n takes $\Theta(n)$ time. With balanced splits, $T(n) = 2T(n/2) + \Theta(n)$, giving $\Theta(n \log n)$ in the best case. At the opposite extreme, a pivot that leaves one side empty gives $T(n) = T(n - 1) + \Theta(n) = \Theta(n^2)$. The last-element rule encounters this pattern on sorted and reverse-sorted distinct inputs. It also occurs on all-equal inputs in both versions because every equal value goes to the left. Random selection cannot improve that duplicate-key case without a different partition method.

For distinct keys in a uniformly random permutation, deterministic Quicksort has $\Theta(n \log n)$ average time. To see why, order keys by rank. Two ranks i and j are compared if the first pivot chosen from the ranks between them is one of those two endpoints, with probability $2/(j - i + 1)$. Summing these probabilities over pairs gives $O(n \log n)$ expected comparisons; partition

overhead has the same order. A comparison-sorting lower bound supplies the matching Ω(n log n) average bound (Cormen et al., 2022).

The partition invariant explains why the procedure sorts correctly. Before each scan step, all scanned values before the boundary are no larger than the pivot, and all scanned values between the boundary and the scan position are larger. When a value is no larger, swapping it into the boundary extends the first group; otherwise the scan simply advances. After the scan, exchanging the pivot with the boundary places it between the groups. Sorting each remaining group therefore gives a sorted interval. An empty or single-element interval requires no further work.

Randomized Quicksort selects uniformly from each active subarray. For any fixed input of distinct keys, this gives the same expected Θ(n log n) bound over the random choices, including for already ordered input. It does not eliminate the Θ(n^2) worst case: repeatedly choosing an extreme pivot remains possible. In fact, for n distinct keys, the probability of an extreme pivot at every nontrivial partition is 2^(n−1)/n!, which falls rapidly as n grows. Expected performance here assumes independent uniform pivot selections; seeded pseudorandom choices make the experiment reproducible.

The two averages describe different sources of uncertainty. For the fixed-pivot version, the stated average assumes all distinct-key input permutations are equally likely; a particular ordered input need not behave like that average. For randomized Quicksort, the input can be fixed, and the expectation is over pivot choices. The implementation uses a pseudorandom generator, not a proof of independence, but seeded runs are useful for repeatable experiments. Randomization reduces sensitivity to input order; it does not guarantee balanced partitions in every run.

The partition uses constant extra storage, and recursing only on the smaller side limits simultaneous recursive depth to O(log n), since each nested call has at most half the elements of its parent. This worst-case auxiliary-space bound excludes the input list and the benchmark's copies. Each recursive call, swaps, comparisons, and random-number generation add practical overhead. A conventional implementation that recursively calls both sides can instead use O(n) stack space on badly unbalanced partitions.

Empirical Comparison

The experiment isolates the pivot rule by keeping partitioning, recursion strategy, input values, and implementation language the same for both algorithms. Each size uses distinct integers, so the sorted and reverse-sorted cases exercise order rather than repeated values. This choice makes the contrast clear but leaves other workloads, especially those with many duplicates, outside the measurements.

The benchmark used Python 3.12.12 on macOS arm64. For each size it used one seeded random permutation, a sorted list, and a reverse-sorted list. Each algorithm ran five times per case on a fresh copy; the table reports the median in milliseconds. Input copying and checking against Python's sorted result were outside the timed interval. Random pivot generators were seeded separately for each run. These are wall-clock measurements, so interpreter, hardware, and competing activity affect the results.

Input	Size	Deterministic ms	Randomized ms
Random	500	0.230	0.306
Random	1,000	0.500	0.695
Random	2,000	1.085	1.466
Random	4,000	2.371	3.060
Sorted	500	4.397	0.256
Sorted	1,000	19.236	0.589
Sorted	2,000	81.242	1.375
Sorted	4,000	325.457	2.788
Reverse	500	3.230	0.257
Reverse	1,000	14.376	0.601
Reverse	2,000	58.248	1.355
Reverse	4,000	234.568	2.841

For the last-element pivot, the sorted input initially selects the largest key and leaves one subarray of size n − 1. Reverse order initially selects the smallest key and also leaves n − 1 items. The same extreme-split pattern continues for these ordered inputs, so the number of scanned elements grows as (n − 1) + (n − 2) + ... + 1. Random pivots can divide the ordered values at many ranks instead; their expected cost depends on the pivot choices rather than the original order.

At 4,000 elements the deterministic version took 325.457 ms on sorted input, versus 2.788 ms randomized. Doubling sorted input from 2,000 to 4,000 increased the deterministic time by about four times, consistent with quadratic growth. On the random permutation at 4,000, the fixed-pivot version took 2.371 ms versus 3.060 ms; random selection adds overhead, although these timings alone cannot isolate its precise cost. The experiment uses one dataset per size and distribution and five repetitions, so it does not capture variation across independent datasets or establish universal performance rankings. It does not measure duplicate-heavy inputs.

Conclusion

The measurements support the theoretical distinction, but they do not prove asymptotic bounds. At 4,000 elements, the reverse case took 234.568 ms with the fixed pivot and 2.841 ms with random pivots. The ordered cases both show a substantial gap, while the random

permutation favors the fixed pivot in this run. Five timing repetitions on one permutation measure runtime variation for that particular dataset, not the full distribution of performance across all permutations. A broader study would use multiple independent inputs and report the spread as well as the median.

Random pivots protect the algorithm in expectation against an input order that consistently defeats a fixed pivot. They do not prevent unlucky choices or fix the quadratic behavior of this two-way partition on equal values. The smaller-side recursion strategy controls stack usage independently of these time costs.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press. https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/